COMPUTER VISION

LABSHEET 8

NAME – HARSH RAJ

CUID – CU240251762

SEC – C , ROLL- 23

```
#NAME - PALLAV PANKAJ
#ROLL - 40


import cv2
import numpy as np
import os

# -------------------- SETTINGS --------------------
VIDEO_PATH = "input.mp4"
USE_WEBCAM = False

MAX_CORNERS = 150
QUALITY_LEVEL = 0.3
MIN_DISTANCE = 7
BLOCK_SIZE = 7

# Lucas-Kanade parameters
LK_WIN_SIZE = (15, 15)
LK_MAX_LEVEL = 2
LK_CRITERIA = (
    cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
    10,
    0.03,
)

# -------------------- VIDEO SOURCE --------------------
if USE_WEBCAM:
    cap = cv2.VideoCapture(0)
else:
    if not os.path.exists(VIDEO_PATH):
        print(f"ERROR: '{VIDEO_PATH}' not found.")
        print("Place an MP4 video named 'input.mp4' in this folder,")
        print("or change VIDEO_PATH in the program.")
        raise SystemExit

    cap = cv2.VideoCapture(VIDEO_PATH)
```

```
if not cap.isOpened():
    print("ERROR: Could not open video source.")
    raise SystemExit

# Read first frame
ret, old_frame = cap.read()
if not ret:
    print("ERROR: Could not read the first frame.")
    cap.release()
    raise SystemExit

# Resize to a reasonable display size
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 360

old_frame = cv2.resize(old_frame, (DISPLAY_WIDTH, DISPLAY_HEIGHT))
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

#NAME - PALLAV PANKAJ
#ROLL - 40


# -------------------- SHI-TOMASI CORNER DETECTION --------------------
feature_params = dict(
    maxCorners=MAX_CORNERS,
    qualityLevel=QUALITY_LEVEL,
    minDistance=MIN_DISTANCE,
    blockSize=BLOCK_SIZE,
)

p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

# Lucas-Kanade trajectory mask
lk_mask = np.zeros_like(old_frame)

# Statistics
frame_count = 0
total_distance = 0.0
previous_center = None
fps = cap.get(cv2.CAP_PROP_FPS)

if fps <= 0:
    fps = 30.0

def detect_features(gray):
    return cv2.goodFeaturesToTrack(gray, mask=None, **feature_params)

def add_title(image, title):
    result = image.copy()
```

```
    cv2.rectangle(result, (0, 0), (result.shape[1], 38), (30, 30, 30), -1)
    cv2.putText(
        result,
        title,
        (12, 27),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.75,
        (255, 255, 255),
        2,
        cv2.LINE_AA,
    )
    return result

def resize_for_display(image):
    return cv2.resize(image, (DISPLAY_WIDTH, DISPLAY_HEIGHT))
#NAME - PALLAV PANKAJ
#ROLL - 40


# -------------------- MAIN LOOP --------------------
while True:
    ret, frame = cap.read()

    if not ret:
        # For a video file, restart from the beginning.
        if not USE_WEBCAM:
            cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
            ret, frame = cap.read()

            if not ret:
                break

            old_frame = cv2.resize(frame, (DISPLAY_WIDTH, DISPLAY_HEIGHT))
            old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
            p0 = detect_features(old_gray)
            lk_mask = np.zeros_like(old_frame)
            previous_center = None
            continue
        else:
            break

    frame = resize_for_display(frame)
    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # ============================================================
    # 1. LUCAS-KANADE SPARSE OPTICAL FLOW
    # ============================================================
    lk_output = frame.copy()
```

```
    if p0 is None or len(p0) < 5:
        p0 = detect_features(old_gray)
        lk_mask = np.zeros_like(frame)

    if p0 is not None:
        p1, st, err = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            winSize=LK_WIN_SIZE,
            maxLevel=LK_MAX_LEVEL,
            criteria=LK_CRITERIA,
        )

        if p1 is not None and st is not None:
            good_new = p1[st == 1]
            good_old = p0[st == 1]

            centers = []

            for new, old in zip(good_new, good_old):
                x_new, y_new = new.ravel()
                x_old, y_old = old.ravel()

                # Draw trajectory
                cv2.line(
                    lk_mask,
                    (int(x_old), int(y_old)),
                    (int(x_new), int(y_new)),
                    (0, 255, 0),
                    2,
                )

                # Draw current feature point
                cv2.circle(
                    lk_output,
                    (int(x_new), int(y_new)),
                    4,
                    (0, 0, 255),
                    -1,
                )

                # Draw displacement vector
                cv2.arrowedLine(
                    lk_output,
                    (int(x_old), int(y_old)),
                    (int(x_new), int(y_new)),
```

```
                    (255, 0, 0),
                    1,
                    tipLength=0.25,
                )

                centers.append([x_new, y_new])

            # Overlay trajectory
            lk_output = cv2.add(lk_output, lk_mask)

            # Estimate object/feature center
            if len(centers) > 0:
                center = np.mean(centers, axis=0)
                cx, cy = int(center[0]), int(center[1])

                cv2.circle(lk_output, (cx, cy), 8, (0, 255, 255), -1)
                cv2.putText(
                    lk_output,
                    f"Center: ({cx}, {cy})",
                    (10, 65),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.5,
                    (255, 255, 255),
                    1,
                    cv2.LINE_AA,
                )

                if previous_center is not None:
                    dx = center[0] - previous_center[0]
                    dy = center[1] - previous_center[1]
                    magnitude = float(np.sqrt(dx * dx + dy * dy))

                    total_distance += magnitude

                    if magnitude > 0.5:
                        angle = np.degrees(np.arctan2(dy, dx))

                        if -22.5 <= angle < 22.5:
                            direction = "RIGHT"
                        elif 22.5 <= angle < 67.5:
                            direction = "DOWN-RIGHT"
                        elif 67.5 <= angle < 112.5:
                            direction = "DOWN"
                        elif 112.5 <= angle < 157.5:
                            direction = "DOWN-LEFT"
                        elif angle >= 157.5 or angle < -157.5:
                            direction = "LEFT"
                        elif -157.5 <= angle < -112.5:
```

```
                            direction = "UP-LEFT"
                        elif -112.5 <= angle < -67.5:
                            direction = "UP"
                        else:
                            direction = "UP-RIGHT"

                        cv2.putText(
                            lk_output,
                            f"Direction: {direction}",
                            (10, 90),
                            cv2.FONT_HERSHEY_SIMPLEX,
                            0.55,
                            (0, 255, 255),
                            2,
                            cv2.LINE_AA,
                        )

                        cv2.putText(
                            lk_output,
                            f"Motion: {magnitude:.2f} px/frame",
                            (10, 115),
                            cv2.FONT_HERSHEY_SIMPLEX,
                            0.5,
                            (255, 255, 255),
                            1,
                            cv2.LINE_AA,
                        )

                    previous_center = center

                p0 = good_new.reshape(-1, 1, 2)
#NAME - PALLAV PANKAJ
#ROLL - 40


    # ============================================================
    # 2. FARNEBACK DENSE OPTICAL FLOW
    # ============================================================
    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        frame_gray,
        None,
        pyr_scale=0.5,
        levels=3,
        winsize=15,
        iterations=3,
        poly_n=5,
        poly_sigma=1.2,
        flags=0,
```

```
    )

    magnitude, angle = cv2.cartToPolar(flow[..., 0], flow[..., 1])

    # HSV visualization:
    # Hue = direction, Value = magnitude
    hsv = np.zeros_like(frame)
    hsv[..., 0] = angle * 180 / np.pi / 2
    hsv[..., 1] = 255
    hsv[..., 2] = cv2.normalize(
        magnitude, None, 0, 255, cv2.NORM_MINMAX
    )

    farneback_output = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    # Draw sparse motion vectors over Farneback result
    step = 25

    for y in range(0, DISPLAY_HEIGHT, step):
        for x in range(0, DISPLAY_WIDTH, step):
            fx, fy = flow[y, x]
            if magnitude[y, x] > 1.0:
                cv2.arrowedLine(
                    farneback_output,
                    (x, y),
                    (int(x + fx * 3), int(y + fy * 3)),
                    (255, 255, 255),
                    1,
                    tipLength=0.3,
                )

    mean_magnitude = float(np.mean(magnitude))

    cv2.putText(
        farneback_output,
        f"Mean motion: {mean_magnitude:.2f}",
        (10, 65),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.55,
        (255, 255, 255),
        2,
        cv2.LINE_AA,
    )

    # ============================================================
    # DISPLAY SIDE-BY-SIDE
    # ============================================================
    lk_display = add_title(
```

```
        lk_output,
        "Lucas-Kanade (Sparse Optical Flow)",
    )

    fb_display = add_title(
        farneback_output,
        "Farneback (Dense Optical Flow)",
    )

    combined = np.hstack((lk_display, fb_display))

    # Bottom information bar
    info_height = 48
    info = np.zeros((info_height, combined.shape[1], 3), dtype=np.uint8)

    cv2.putText(
        info,
        "Q/ESC: Quit   R: Re-detect features",
        (15, 31),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.65,
        (255, 255, 255),
        2,
        cv2.LINE_AA,
    )

    combined = np.vstack((combined, info))

    cv2.imshow(
        "Experiment 8 - Optical Flow Object Tracking",
        combined,
    )

    # Update previous frame
    old_gray = frame_gray.copy()

    frame_count += 1

    # Key controls
    key = cv2.waitKey(20) & 0xFF

    if key == ord("q") or key == 27:
        break

    if key == ord("r"):
        p0 = detect_features(old_gray)
        lk_mask = np.zeros_like(frame)
#NAME - PALLAV PANKAJ
```

```
#ROLL - 40



# -------------------- RESULTS --------------------
elapsed_time = frame_count / fps if fps > 0 else 0
average_speed = total_distance / frame_count if frame_count > 0 else 0

print("\n========== EXPERIMENT 8 RESULTS ==========")
print("Application: Optical Flow Object Tracking")
print(f"Frames processed      : {frame_count}")
print(f"Approx. video time    : {elapsed_time:.2f} seconds")
print(f"Total tracked motion  : {total_distance:.2f} pixels")
print(f"Average motion        : {average_speed:.2f} pixels/frame")
print("Methods used          : Lucas-Kanade and Farneback")
print("==========================================\n")

cap.release()
cv2.destroyAllWindows()



#NAME - PALLAV PANKAJ
#ROLL - 40


# 1. How does object tracking differ from object detection?

# Object detection identifies the location and class of an object in an image
or video frame. Object tracking follows the same object across consecutive
frames and estimates its movement and trajectory. Tracking therefore maintains
the object's position over time.

# 2. Explain how Optical Flow can be used for real-time object tracking.

# Optical Flow estimates the apparent motion of pixels between consecutive
video frames. Feature points can be detected using Shi-Tomasi and then tracked
using the Lucas-Kanade algorithm. The movement of these points gives the
object's displacement, direction, and trajectory.

# 3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade
Optical Flow algorithm?

# Shi-Tomasi detects strong corner or feature points in an image. These points
are suitable for tracking because their locations can be reliably identified
in successive frames. Lucas-Kanade then calculates how these selected points
move between frames.

# 4. Why is Optical Flow suitable for motion analysis in videos?
```

```
# Optical Flow estimates motion between consecutive frames, allowing a system to determine the direction, magnitude, displacement, trajectory, and movement pattern of objects. This makes it useful for continuous video-based motion analysis.

# 5. What challenges arise while tracking fast-moving or partially occluded objects?

# Fast-moving objects can cause large changes in position between frames, making feature tracking difficult. Partial occlusion can hide important feature points. Illumination changes and camera movement can also reduce tracking accuracy.

# 6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.
# Optical Flow  Deep Learning Tracking
# Estimates pixel/feature motion    Uses trained neural networks
# Usually computationally lighter   Generally computationally heavier
# Does not necessarily require object-class training    Often requires training data
# Good for motion estimation    Better for complex object recognition/tracking
# Sensitive to illumination and camera movement Can be more robust to complex scenes
# 7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

# Optical Flow can estimate the movement and direction of vehicles, pedestrians, and other objects. In traffic monitoring it can help analyze vehicle movement and traffic patterns. In autonomous driving, motion information can assist with understanding the movement of surrounding objects. Traffic monitoring and autonomous navigation are specifically identified as applications of optical flow in the experiment.

# 8. Explain the effect of camera motion on Optical Flow estimation.

# Camera movement causes many or most pixels in the scene to appear to move, even when the objects themselves are stationary. Therefore, the calculated optical flow can contain motion caused by the camera rather than the objects. This can reduce tracking accuracy and make object motion analysis more difficult.

# 9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

# Five applications are:

# Object tracking
# Traffic monitoring
```

```
# Human activity recognition
# Autonomous navigation
# Video surveillance

# Sports analytics is another application mentioned in the experiment.

# 10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

# Optical Flow provides information about how objects and people move over time. In surveillance, it can help identify and track moving objects. In robotics, motion information can support navigation and interaction with the environment. In human activity recognition, movement patterns can be analyzed to identify activities.
```

OUTPUT

```
========== EXPERIMENT 8 RESULTS ==========
Application: Optical Flow Object Tracking
Frames processed      : 76
Approx. video time    : 3.17 seconds
Total tracked motion  : 57.05 pixels
Average motion        : 0.75 pixels/frame
Methods used          : Lucas-Kanade and Farneback
==========================================
```